

Week 1 Lesson Plan

Enterprise RAG: When Meaning Becomes Geometry

ASIF QAMAR



pepal leaves publisher

www.supportvectors.ai

All rights reserved. The contents in this document are the intellectual property of SupportVectors. No part of it can be shared at all without the explicit permission of the author.

First draft, June 6, 2026

When Meaning Becomes Geometry

What you seek is seeking you.

Rumi

Not everything that can be
counted counts, and not
everything that counts can be
counted.

attributed to William Bruce
Cameron

A Gentle Re-Entry

WELCOME. Some of you sat with us two evenings ago at the public talk, *From RAG to Riches*, and watched a machine retrieve a photograph of a dog when asked, in words, about *learning to love an animal*. Others arrive fresh, drawn by the promise that one can build systems which reason over a corpus of private documents rather than over the frozen, second-hand memory baked into a model's weights. Wherever you are coming from, today we begin the same way: not by building, but by *orienting*.

A subject as broad as enterprise-scale search and Retrieval-Augmented Generation can feel like a mountain range seen for the first time. The temptation is to start climbing immediately—to reach for a vector database and a framework and a clever prompt. We will resist it. In this first session we learn where the mountains are, why they exist, and which passes are treacherous. Mastery comes later, and it comes hands-on: over the next three months you will build retrieval pipelines,

This lesson plan is also your refresher. Read it first for the shape of the argument; return to the detailed book chapters when you need the full derivations. When you are short on time before a later week, come back *here*, not to the chapters. The plan is dense by design—it is meant to repay re-reading.

chunkers, re-rankers, query rewriters, guardrails, grounding systems, and a structured-data bridge, and you will measure all of it against real metrics. But a system you build without understanding *why* it behaves as it does is a system you cannot debug at midnight when it confidently returns the wrong contract clause to a regulator. So we begin with the worldview on which everything else rests.

HERE IS THE SINGLE THREAD that will run through the entire day, and indeed the entire course. Modern machines do not reason in words. They reason in *vectors*—points in a high-dimensional space where the geometry itself encodes meaning. Similar thoughts land in nearby neighbourhoods; unrelated ones drift apart. Search, in this worldview, is not string-matching. It is *navigation through a space of meaning*. And the same is true one level deeper: the knowledge a language model appears to “have” is also geometry—directions and features laid down in its weights, which, as we will see, we can now literally query. Every later engineering decision we make—how we cut documents, how we retrieve, how we re-rank, how we ground, how we evaluate—is downstream of the geometry of these spaces. Understand the geometry, and the rest of the course becomes a series of sensible responses to its quirks.

WHY? Today sets the compass for twelve weeks. If you leave with only one conviction, let it be this: *meaning has become geometry*, and retrieval is the art of moving well through that geometry. Everything that looks like a clever trick later in the course—hybrid search, re-ranking, RAPTOR trees, graph communities, semantic caches, grounding judges—is a disciplined response to a geometric fact you will meet for the first time today.

What Must You Carry Forward

WHEN TODAY’S SESSION CONCLUDES, you should be able to articulate—without hand-waving—each of the following:

- Explain why organisations fail not from a lack of information but from a lack of *retrievability*, and why retrieval is the epistemic bottleneck of any RAG system.
- Contrast lexical (keyword) search with semantic search, and give at least three queries on which they diverge sharply—abstraction, emotion, and intent—and explain the bi-encoder picture that makes semantic search possible.
- Distinguish the model that *generates* an answer (a decoder, an LLM) from the model that *embeds* text for retrieval (an encoder), and explain why a RAG system almost always uses two different models.
- Sketch the seq2seq encoder–latent–decoder scaffold, write the autoregressive factorisation of sequence probability, and locate where

attention enters the story—first as Bahdanau’s alignment, then as the transformer’s central organ.

- State the scaled dot-product attention equation and read it as a soft, differentiable lookup over keys and values.
- Explain what it means to say machines “think in machineese”: that proximity, similarity, and probability—not symbols and grammar—are the native currency of machine cognition, and connect this to the distributional hypothesis.
- Distinguish *static* embeddings (one vector per word) from *contextual* embeddings (a vector per occurrence), and explain why polysemy forces the move from the first to the second.
- Describe, at an introductory level, what it means to *query into* a model’s weights—features, relations, the residual stream—and why this makes the “black box” less black.
- Define an embedding as a point in $\mathbb{R}^d$, write cosine similarity, and explain what cosine rewards that the dot product does not.
- State the *curse of dimensionality* and the phenomenon of *concentration of measure*, explain why random high-dimensional vectors are nearly orthogonal, and explain why “nearest neighbour” nonetheless remains meaningful.
- Distinguish concentration of measure (a property of the *space*) from anisotropy (a property of the *learned distribution* of embeddings), and explain why conflating the two produces wrong intuitions.
- Explain, qualitatively, why raw BERT embeddings occupy a narrow cone (anisotropy) while embeddings trained for retrieval spread out and separate.
- List the principal pathologies of *naive* RAG and connect each one to a geometric fact established earlier in the day.

If you can do these, you are ready for Week 2, where we make our first irreversible engineering decision: how to cut a document into retrievable pieces.

Act I: The Magic, and the Map

EVERY GOOD JOURNEY OPENS WITH A REASON TO TAKE IT. Act I is that reason. We watch search behave in a way that ought to be impossible for a string-matching machine, we ask how organisations lose what they already know, and we lay out the map of the fourteen instruments we will build before the summer ends.

Search as Civilizational Infrastructure

SEARCH IS USUALLY INTRODUCED AS A SOLVED PROBLEM. We type words; a machine returns documents; we move on. Yet whole organisations stumble not because they lack information but because they cannot *retrieve* what they already possess. The contract that was signed, the experiment that already failed, the customer who already complained—all of it sits in some store, indexed by words that no longer match the question being asked. Knowledge an organisation cannot find is, for every practical purpose, knowledge it does not have.

Will Durant observed that civilisation is a stream with banks: the stream carries the deeds we record, but the banks—the quiet, unremembered work of keeping and retrieving knowledge—are what let each generation begin where the last left off. Retrieval is bank-work. It is unglamorous and it is load-bearing. A company’s collective memory is only as useful as its retrievability, and the same is true of a language model: a model that cannot reach the right fact at the right moment will confidently invent one.

It helps to name the two ancient failure modes precisely, because the whole course oscillates between them. *Precision* asks: of the things I returned, how many were relevant? *Recall* asks: of the relevant things that existed, how many did I return? A search that returns nothing has perfect precision and useless recall; a search that returns everything has perfect recall and useless precision. Every retrieval system is a negotiated truce between these two, and much of the engineering ahead—hybrid search, re-ranking, thresholds—is about moving the truce to where your application needs it.

So we begin from a premise different from the usual one. Search is not a technical convenience bolted onto a chatbot. It is the interface between human intent and collective memory, and Retrieval-Augmented Generation is the architectural response to a hard limit of unaided generation: a model’s parametric memory is frozen at training time, finite in capacity, expensive to update, and unable to cite its sources. RAG hands the model an open book at the moment of answering—and the art of the course is making sure the book is open to the right page.

This is why we treat retrieval as the *epistemic bottleneck* of RAG. A generator can only be as truthful as the evidence placed before it. Bad retrieval caps the quality of everything downstream—no prompt, however clever, and no larger model, however capable, repairs evidence that was never fetched.

A Demonstration of Semantic Search

LET US WATCH THE THING BEFORE WE EXPLAIN IT. In the companion notebook—the same one we used at the public talk—we pose queries to a collection of images and passages and watch what comes back. We ask, in words, about *learning to love an animal*, and the system returns a photograph of a child with a dog, though the word “dog” never appears in the query and no caption was matched. We ask for something *melancholy*, or *hopeful*, and the retrieved items share a mood rather than a keyword. We ask, abstractly, about *regulatory pressure on a company*, and back come passages about FDA letters and EMA guidelines that never use our words at all.

This should be unsettling if you still picture search as string-matching. No substring of “to love an animal” occurs in the pixels of a photograph. Something has been matched, but it is not text. What has been compared is *meaning*, rendered as position in a space.

Here is the mechanism in one breath, to be unpacked all day. An encoder turns the query into a vector $\mathbf{e}_q$; the same kind of encoder has already turned every document into a vector $\mathbf{e}_d$; all of these live in one shared space; and we retrieve the documents whose vectors are closest to the query’s. When the query is text and the documents are images, we use a model trained to place *both* modalities in one shared space—the trick behind CLIP-style image–text retrieval— which is why words can find pictures.

We will return, again and again, to three families of query on which lexical and semantic search diverge. *Abstraction*: a query about “regulatory pressure” matching a passage that never uses those words but is plainly about it. *Emotion*: matching a shared mood across very different vocabulary. *Intent*: matching what the user is trying to accomplish rather than what they literally typed. These divergences are not curiosities. They are the reason the rest of the course exists—and also, as we will see in Act III, the seed of how semantic search goes wrong.

The picture to hold: a *bi-encoder*. One encoder maps the query to a vector; another (often the same) maps each document to a vector; both land in the *same* space; retrieval is finding the document vectors closest to the query vector. Dense Passage Retrieval is the canonical trained version of exactly this idea, and we will build one.

The Map: Fourteen Instruments, One Pipeline

BEFORE WE DESCEND INTO MECHANISM, let us see the whole route. The course is a single pipeline—from a raw document to a measured, trustworthy answer—and each week adds one instrument to it.

Foundations How meaning becomes geometry (today); how the machine learns to place it there.

Hold this map loosely for now. By the end of today you will understand *why* the very first instruments exist: the pathologies of naive RAG that we meet in Act III are precisely the problems these fourteen instruments were built to solve. The syllabus is not a list; it is an argument, and today is its first premise.

Ingestion Chunking—cutting documents into retrievable units, the first and most irreversible projection.

Retrieval The sparse/dense/hybrid pipeline and its cascade of re-rankers; query transformation, the translator that rewrites the question before it is ever asked.

Enrichment Derivative artifacts (the prism), RAPTOR (the zoom lens), and GraphRAG (the telescope)—three ways of seeing one corpus at different resolutions.

Architecture & Speed How scale decides architecture; semantic caching, the system’s memory for questions it has already answered.

Safety & Trust Request guardrails (the gatehouse before the castle) and response grounding (the judge after the jury).

Proof & Frontier Evaluation—the measure of all things—and the bridge to structured data via Text-to-SQL.

Notice the shape. We do not begin with frameworks and APIs. We begin with a worldview, because the instruments only make sense once you can see the geometry they are operating on. That worldview is the work of Act II.

Act II: How the Machine Sees — From Words to Vectors

IF SEMANTIC SEARCH COMPARES MEANING, then meaning must have been turned into something a machine can compare. Act II tells that story: what a language model is, the discovery that let it scale, the central claim that machines reason in vectors, the problem of words with many meanings, and a first, startling look at querying a model’s knowledge directly.

What Is a Language Model? The Seq2Seq Scaffold

STRIP AWAY THE MARKETING and a language model is a *sequence model*: a function that, given a sequence of tokens, assigns probabilities to what comes next. Formally, it factorises the probability of a whole sequence into a product of next-token probabilities,

$$p(y_1, y_2, \dots, y_T) = \prod_{t=1}^T p(y_t \mid y_1, \dots, y_{t-1}),$$

and generation is *autoregressive*: predict y_t , append it, condition on it, predict y_{t+1} . Each factor is a distribution over the entire vocabulary, and “writing” is repeatedly sampling from it.

The pedagogically useful scaffold for relating an input to an output is *sequence-to-sequence*, introduced by Sutskever and colleagues: an *encoder* reads the input and compresses it into an internal representation—a latent $\mathbf{z}$ —and a *decoder* expands that latent into an output sequence.

$$\text{input} \xrightarrow{\text{encoder}} \mathbf{z} \xrightarrow{\text{decoder}} \text{output}.$$

Read it as a pipeline of meaning: the encoder’s job is to *understand* (to map text into a latent that captures its content), and the decoder’s job is to *speak* (to turn a latent back into fluent text). Early machine-translation systems used exactly this shape—read the French, form a thought, write the English.

A distinction worth fixing now, because it will save you weeks of confusion. The model that *generates* your answer (a decoder-style LLM) is, in almost every production RAG system, *not* the same model that *embeds* your documents for retrieval (an encoder-style embedding model). The generator is eloquent and expensive and runs once per answer; the embedder is silent and cheap and has already run over every chunk in your corpus, placing each as a point in a space so that similar texts land nearby. When we say “the embedding model” in this course, we mean the encoder. When we say “the LLM” or “the generator,” we mean the decoder. Conflating them is the single most common Week-1 misconception, and it quietly corrupts one’s reasoning about cost, latency, and failure modes for weeks.

This factorisation is why a language model can be *steered* by context: everything to the left—including the documents RAG inserts—conditions every probability that follows. Retrieval works precisely because the model attends to the evidence we place in $y_1, \dots, y_{t-1}$.

The bottleneck in the original seq2seq design is that *all* the meaning of a long input had to squeeze through a single fixed-size latent $\mathbf{z}$. For a short sentence, fine; for a paragraph, the latent forgets its own beginning by the time it reaches the end. This bottleneck is exactly what attention was invented to relieve.

Attention: From Bahdanau’s Alignment to “All You Need”

THE KEY DISCOVERY that made modern representations possible has a two-part history, and both parts are worth telling. The first is Bahdanau, Cho, and Bengio’s *alignment* mechanism. Instead of forcing the whole input through one latent, they let the decoder, at each output step i , form a *weighted average* of all the encoder’s hidden states h_j :

$$c_i = \sum_j \alpha_{ij} h_j, \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})},$$

where the weights α_{ij} are a softmax over learned *alignment scores* e_{ij} measuring how relevant input position j is to output position i . Attention began life, in other words, as a cure for the seq2seq bottleneck: let

The $\sqrt{d_k}$ rescaling is our first quiet encounter with high-dimensional geometry: dot products of long vectors grow with dimension, and without rescaling the softmax would saturate. We will meet the same “things get big and concentrated in high dimensions” theme, far more dramatically, in Act III.

We will not derive multi-head self-attention in full today—it is the engine of *generation* more than of *retrieval*, and we will return to it. Today it is enough to see where it sits in the lineage: alignment fixed the bottleneck; the transformer made alignment the whole machine. We linger instead on the geometry it produces.

Demo. Poloclub’s Transformer Explainer, poloclub.github.io/transformer-explainer. Treat it as a window, not a lecture: see the shapes of the tensors and the flow of attention, then return to our thread.

the decoder *look back* and attend more to the input words that matter for the word it is about to produce.

The second part is the leap. Vaswani and colleagues asked a radical question: if attention is so powerful, what if we throw away the recurrence entirely and build the whole model out of attention? The resulting architecture—the *transformer*—is organised around *scaled dot-product attention*,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Read this as a *soft, differentiable lookup*. Each token emits a *query* Q (“what am I looking for?”), every token offers a *key* K (“what do I match?”) and a *value* V (“what do I contribute if matched?”). The dot product $QK^\top$ scores every query against every key; the $\sqrt{d_k}$ keeps those scores from exploding as dimension grows; the softmax turns them into attention weights; and the weighted sum of values is what each token carries forward. Doing this with several independent “heads” in parallel (*multi-head* attention) lets the model match on several kinds of relationship at once, and because every position is processed in parallel rather than in sequence, the architecture scales to enormous data and parameter counts. Its title, *Attention Is All You Need*, was not hyperbole; it was a thesis, and the last several years have largely vindicated it.

To make the abstraction tangible, we open the *Transformer Explainer* (the Poloclub interactive visualisation) and watch a real transformer turn a prompt into a distribution over next tokens—attention weights lighting up between words, the residual stream carrying information forward layer by layer. The point is not to master the internals; it is to see, with your own eyes, that the thing operates on *vectors*—and that brings us to the central claim of the day.

Machines Think in “Machinese”

HERE IS THE CLAIM, STATED PLAINLY. Modern models do not manipulate words, letters, or grammar rules. The first thing that happens to your text is *tokenisation*—it is chopped into sub-word units, each mapped to an integer, each integer mapped to a learned vector. From that point on, every token, every passage, every image patch is a *vector*: an ordered list of real numbers, a point in a high-dimensional space. Tensors—stacks of such vectors—are the only objects the machine ever truly handles. The native currency of machine cognition is not symbol and syntax; it is *proximity, similarity, and probability*.

Why should meaning survive this translation into numbers? Because

of a principle that predates deep learning by decades: the *distributional hypothesis*—“you shall know a word by the company it keeps” (Firth, 1957; Harris, 1954). Words that occur in similar contexts come to occupy similar positions. Mikolov and colleagues showed this concretely with word2vec, and Pennington and colleagues with GloVe: not only do related words cluster, but *relationships* appear as consistent *directions* in the space—the famous king – man + woman $\approx$ queen.

If meaning is geometry, then similarity must be a geometric quantity. The two measures we use constantly are the *dot product* and *cosine similarity*:

$$\text{sim}_{\cos}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \cos \theta.$$

Cosine similarity measures the *angle* between two vectors and ignores their lengths; it asks “do these point the same way?” The dot product keeps the lengths and so also rewards magnitude. For most semantic retrieval we care about direction—the *topic*—more than magnitude, which is why cosine is the workhorse; and note that if we first normalise every vector to unit length, the dot product *becomes* cosine, so the two views coincide. Keep this picture: to retrieve is to find the vectors whose directions align most closely with the direction of the query.

Polysemy: One Word, Many Points

NOW A WRINKLE THAT THE EARLY EMBEDDINGS COULD NOT IRON OUT. Words have more than one meaning. A *bank* is a place for money and also the edge of a river; a *monarch* is a sovereign and also a butterfly; *spring* is a season, a coil, and a source of water. This is *polysemy*, and it is not a rare edge case—the most common words are the most polysemous.

The first generation of embeddings (word2vec, GloVe) was *static*: one vector per word *type*, computed once and frozen. Such a vector is forced to be a compromise—the single point for “bank” is dragged toward both “money” and “river” and ends up stranded between them, near neither. For retrieval this is poison: a query about river ecology and a query about interest rates would both match the same muddled “bank” vector.

The fix is *contextual* embeddings. A transformer encoder (BERT and its descendants) reads the *whole sentence* before placing any token, so the vector for “bank” in “*she sat on the river bank*” lands far from the vector for “bank” in “*she deposited it at the bank*”. Attention is what makes this possible: each token’s representation is a weighted blend of its neighbours, so context literally moves the point. Meaning is no

This is the moment to let the strangeness land. An analogy is not stored as a rule; it is recovered as *vector arithmetic*. “Royalty” is a direction; “gender” is another direction; meaning has become geometry, and reasoning has become moving along directions. The rest of the course is a long meditation on the consequences.

This is exactly why we do not use raw word2vec for enterprise search. Sentence-BERT and its descendants produce *contextual*, sentence-level embeddings whose cosine similarity is actually meaningful—the move from a single muddled point to a context-resolved one.

longer a property of a word; it is a property of a word *in a place*.

There is a deeper and more startling version of this idea, which we flag now and your instructor will develop when we turn to mechanistic interpretability. Inside a model, individual neurons are themselves *polysemantic*: a single neuron fires for several unrelated concepts. The leading explanation is *superposition*—models pack far more features than they have dimensions by assigning features overlapping, non-orthogonal directions, tolerating a little interference in exchange for enormous capacity. Linguistic polysemy (one word, many senses) and mechanistic polysemanticity (one neuron, many features) are two faces of the same pressure: meaning is dense, dimensions are finite, and the geometry must economise. Recent work on *dictionary learning* tries to disentangle these superposed features back into interpretable, “monosemantic” directions.

An Embodied Exercise: The Monarch Cards

ABSTRACTION STICKS BETTER WHEN THE BODY JOINS IN. We do the *Monarch* exercise we ran two evenings ago. Each of you holds a card bearing a word or short phrase—some about a monarch butterfly’s migration, some about a king and his court, some about royalty in chess, some about a music label, a brand, a moth. The word “monarch” is gloriously polysemous; its meaning is fixed only by the company it keeps.

We then arrange ourselves physically by *meaning*: the butterfly cards drift toward one corner, the royalty cards toward another, and the genuinely ambiguous cards find themselves pulled between clusters, unsure where to stand. What we have built, with our bodies, is a low-dimensional embedding space—neighbourhoods of meaning, with the polysemous cards living on the boundaries.

The exercise plants three intuitions we will formalise in Act III: that meaning is *position*, that similarity is *distance* (or angle), and that ambiguity is *being near several clusters at once*. It also plants a worry. When we stood in a room, “nearby” was obvious. In a space of hundreds or thousands of dimensions, is “nearby” still meaningful? That worry is well-founded, and it is where Act III goes.

Querying Into the Model: A First Look at LARQL

WE HAVE SAID THAT MEANING IS GEOMETRY IN THE EMBEDDING SPACE. Here is the provocation that closes Act II: meaning is geometry *inside*

The discomfort of the ambiguous card-holder is the lesson of Section made physical. A static embedding leaves that person stranded forever between clusters; a contextual embedding reads the rest of their card—the *sentence*—and walks them firmly into one neighbourhood. Context resolves polysemy by moving the point.

the model's own weights, too—and we can now query it almost as we would a database. This is the doorway to *mechanistic interpretability*, the study of what the computations inside a model actually represent, which your instructor will open further.

Consider LARQL, whose slogan is deliberately startling: *the model is the database*. It “decompiles” a transformer’s weights into a queryable structure—a *vindex*, or vector index—in which the model’s feed-forward gate vectors become a nearest-neighbour index, its embeddings become a token lookup, and its output projections become labelled edges between concepts. You then pose questions in a small query language, LQL. Ask it to describe an entity and it answers with the relations the model has internalised:

```
larql> DESCRIBE "France";
France
  capital    -> Paris      L27
  language   -> French     L24
  continent  -> Europe     L25
  borders    -> Spain      L18
```

More striking still, one can *trace* how an answer forms as the computation flows through the layers—the *residual stream*. Asking the model to complete “The capital of France is,” LARQL shows the token “Paris” sitting at rank 50 around layer 22, then leaping to rank 1 at layer 24—a visible *phase transition* where the model “decides.” The knowledge was always present as geometry in the weights; the trace lets us watch it crystallise into a prediction.

We raise this on day one for a reason that goes to the heart of the course. It is tempting to treat a language model as an oracle—an opaque box that emits text. But a RAG engineer who believes the box is opaque will treat its failures as magic and its fixes as superstition. The truer picture, and the one this course adopts, is that a model’s apparent knowledge is structured, located, and—increasingly—inspectable. When we later build grounding pipelines that check whether an answer is supported by evidence, and guardrails that decide what the model should never answer, it will matter enormously that you think of the model not as a wizard but as a queryable, fallible store of geometric facts.

Act III: The Geometry of Meaning — and Why Naive RAG Breaks

WE NOW TAKE THE INTUITIONS OF ACT II AND LOOK HARD AT THE SPACE ITSELF. High-dimensional spaces are strange places; their strangeness is

Demo / optional tool. LARQL runs without a GPU because it treats stored weights as data to be browsed rather than a network to be run. Notice the layer tags (L27, L24): the knowledge is not in one place but spread across depth, a fact that mechanistic interpretability takes very seriously.

Do not over-interpret a single demo. These tools are young, and a tidy France → Paris edge is a best case, not a guarantee; superposition (Section) means most features are entangled, not clean. The lesson is directional: the black box is becoming less black, and that is good news for anyone who must trust it in production.

not a nuisance to be ignored but the very thing that explains why naive retrieval fails and why the course needs fourteen instruments rather than one.

Embeddings as Points in High-Dimensional Space

FORMALLY, AN EMBEDDING MODEL is a function f that maps a piece of text x to a vector in $\mathbb{R}^d$:

$$f : \mathcal{X} \rightarrow \mathbb{R}^d, \quad x \mapsto \mathbf{e}_x.$$

Typical d ranges from a few hundred to a few thousand. Each coordinate is not a human-interpretable feature; meaning is distributed across all of them, holographically. What matters is the *arrangement*: the relative positions of points, the neighbourhoods, the directions.

Retrieval, in this picture, is nearest-neighbour search. Given a query embedding $\mathbf{e}_q$, we want the documents that maximise $\text{sim}(\mathbf{e}_q, \mathbf{e}_d)$. When the corpus is large, computing this against every document is too slow, so we lean on *approximate nearest neighbour* (ANN) structures—inverted file indexes, product quantisation, and small-world graphs such as HNSW—a computational reality first made rigorous in the work of Indyk and Motwani on similarity search in high dimension.

We will spend Week 2 onward worrying about *what* we embed (whole documents? sentences? overlapping windows?) and *how* we index for speed. Today we worry about something more fundamental: whether the geometry behaves the way our intuition expects. It does not.

The Curse of Dimensionality

OUR INTUITIONS ABOUT DISTANCE were trained in three dimensions, and they betray us in three hundred. The umbrella term for these betrayals is the *curse of dimensionality*: as d grows, volume grows so explosively that any finite set of points becomes hopelessly sparse, and several quantities we rely on begin to behave counterintuitively.

A vivid example. Inscribe a unit ball inside a unit cube and ask what fraction of the cube the ball fills. The volume of a d -dimensional ball of radius r is

$$V_d(r) = \frac{\pi^{d/2}}{\Gamma\left(\frac{d}{2} + 1\right)} r^d,$$

and as d grows this fraction *collapses toward zero*. In two dimensions the disc fills about 79% of the square; in three, the ball fills about 52% of

The dimensionality d is best read as *representational capacity*: more dimensions, more room to keep distinct meanings apart. But capacity is not free. As d grows, our Euclidean intuitions—built in three dimensions—begin to mislead us, sometimes badly. The next three subsections are about those misleadings, and about why retrieval survives them anyway.

the cube; by a few dozen dimensions, the ball fills a vanishing sliver. Almost all the volume of the cube has fled into its corners, far from the centre. Volume, in high dimension, lives at the edges.

The practical upshot is twofold. First, naive exhaustive search becomes expensive and naive spatial indexes stop helping, motivating the approximate methods we will study. Second—and more subtly—the very notion of a “nearest” neighbour comes under pressure. To understand why, and why it nevertheless survives, we need the most beautiful idea of the day.

Concentration of Measure

IN HIGH DIMENSIONS, RANDOMNESS BECOMES ODDLY ORDERLY. Draw many points at random in $\mathbb{R}^d$ and measure all their pairwise distances. In low dimension these distances vary widely. In high dimension they *concentrate*: almost all pairs end up at nearly the *same* distance from one another, and the histogram of distances collapses toward a single sharp spike.

Two precise faces of this phenomenon are worth stating. First, for many natural distributions the distance of a random point from the mean concentrates tightly around its expected value—the relative spread shrinks as d grows:

$$\frac{\text{std}(\|\mathbf{X}\|)}{\mathbb{E}[\|\mathbf{X}\|]} \rightarrow 0 \quad \text{as } d \rightarrow \infty.$$

This is the engine behind the “thin shell”: a high-dimensional Gaussian puts almost none of its mass at the centre and almost all of it in a thin spherical shell at radius $\approx \sqrt{d}$. Second, and just as important for us, two *independent* random vectors in high dimension are very nearly *orthogonal*—their expected cosine similarity is 0 with variance on the order of $1/d$, so as d grows a random pair sits ever closer to a right angle.

Here is the resolution, stated carefully because it is easy to get wrong. Concentration of measure says that *uniform randomness* in high dimension is nearly featureless. It does *not* say retrieval is impossible. It says that any useful structure must be *put there by learning*, and that the margin between a true match and a random one can be thin—which is exactly why re-ranking, hybrid search, and careful evaluation (later instruments) earn their keep. The curse describes the space we are given; learning is how we make it habitable; retrieval lives in the thin margin learning carves out.

Why this matters for us: the space is mostly empty, and emptiness has consequences for how *distances* distribute—which is the subject of concentration of measure, next. It also means brute-force geometric shortcuts that work in low dimensions (grids, k-d trees) degrade, motivating the approximate indexes mentioned above.

The near-orthogonality of random directions is a double-edged gift. It is *good*: it means a high-dimensional space can hold an enormous number of nearly independent directions, which is exactly the capacity that lets models pack many features in (recall superposition, Section). It is also *sobering*: if random things are at 90° and roughly equidistant, then the *margin* that separates a true match from the background crowd must be carved in deliberately by learning.

Anisotropy versus Isotropy: A Demonstration

NOW A SUBTLE AND CRUCIAL DISTINCTION, and the one most often muddled. Concentration of measure is a property of the *space*—it would hold for points scattered uniformly at random. *Anisotropy* is a property of the *learned distribution* of real embeddings *within* that space, and the two are not the same. A distribution is *isotropic* if it spreads roughly evenly in all directions; it is *anisotropic* if it crowds into a narrow region—say, a thin cone—leaving most directions unused.

One clean way to measure it: take many *random pairs* of real embeddings and average their cosine similarity. In a well-behaved (isotropic) space this average should sit near zero, echoing the near-orthogonality of the previous section. For raw, off-the-shelf contextual embeddings it does not: the average cosine of unrelated texts comes out markedly positive, because every vector points in roughly the same general direction. Equivalently, the covariance of the embedding cloud is dominated by a few large eigenvalues—a few directions soak up most of the variance. This is the *representation degeneration* that afflicts naive embeddings, and its consequence is brutal: when everything points the same way, “most similar” loses its power to discriminate.

We make this visible. We open Google’s *Embedding Projector* on a small dataset with three subjects—physics, biology, and history. With raw BERT embeddings, something disappointing happens: the three subjects do not separate. Instead the whole cloud is squeezed into a narrow, funnel-like cone, so even unrelated passages show high cosine similarity. Switch to embeddings from a model trained specifically to place semantically distinct texts apart, and the cloud opens up: physics, biology, and history pull into three visibly separated regions, and finer structure appears within each. The space has become more *isotropic* and, crucially, more *separated*.

The lesson for a practitioner is immediate and sobering. An embedding model is not a neutral ruler. Its *geometry*—how isotropic, how separated—determines whether your retrieval can tell a physics passage from a history one at all. Choosing, and later adapting, the embedder is therefore among the highest-leverage decisions in the whole pipeline, not a default to be accepted unexamined. Much of the engineering ahead exists because real embedding spaces are messier than the tidy isotropic ideal.

Demo. The TensorFlow Embedding Projector with our three-subject corpus. Compare raw transformer-encoder embeddings against embeddings from a model trained for retrieval, and watch the *shape* of the cloud change.

We will deliberately *not* open the hood on the training objective today. The question to sit with is the right one to ask of any phenomenon: *why should there be this difference at all?* Why would one model crush meaning into a cone while another spreads it across the sphere? Hold the question. We will earn its answer in a later week—it has to do with what each model was *asked* to do during training, and with the difference between predicting masked words and telling pairs of sentences apart.

Naive RAG and Its Pathologies

WE CAN NOW ASSEMBLE THE SIMPLEST POSSIBLE RAG SYSTEM—and watch the geometry we have studied predict exactly how it fails. *Naive RAG* is the three-line recipe: embed every chunk of every document and store the vectors; at query time, embed the question and take the top- k nearest chunks; paste them into the prompt and let the LLM answer. It is a fine first prototype and a poor production system, and the reasons trace directly back to Acts II and III.

Semantic near-misses. Because anisotropy compresses the space and concentration thins the margin, the top- k “nearest” chunks may be only marginally nearer than irrelevant ones. The system retrieves plausible-but-wrong neighbours and the generator, trusting its context, states them confidently. This is the geometry of Sections and biting in production.

Polysemy collisions. A query about one sense of a word retrieves chunks about another (Section)—the river bank when you meant the financial one—unless context is preserved and representations are good.

Lost in the cut. A fact split across a chunk boundary is unretrievable; no embedding can recover information destroyed at ingestion. (Week 2: chunking.)

Lexical blindness. A rare identifier—a part number, a statute, a gene—may carry little semantic signal and be missed entirely by dense search, even though a keyword index would have nailed it. (Hybrid retrieval.)

No re-ranking. With matches concentrated near the noise floor, first-pass similarity is too blunt to order the survivors well. (The re-ranking cascade.)

Garbled questions. A misspelled, contextless, or multi-hop question embeds to the wrong place before retrieval even begins. (Query transformation.)

Ungrounded answers. Nothing checks that the generated answer is actually supported by the retrieved evidence. (Guardrails and grounding.)

Unmeasured. Worst of all, nothing tells us *which* of these is hurting us. Without evaluation, every fix is a guess. (The measure of all things.)

This is the bridge we have been building toward all day. The strangeness of high dimension is not academic. It is the mechanism behind every

Look back at the map in Section . Each pathology above is an instrument waiting to be built. This is the promise of the course made concrete: the fourteen instruments are not a grab-bag of techniques but a precise, point-by-point response to the ways naive RAG breaks.

failure of naive RAG, and the justification for every instrument we will build. Understand the geometry, and the syllabus stops looking like a list and starts looking like an argument.

What You Will Build: The Capstone Projects

THEORY EARNs ITS KEEP WHEN IT SHIPS. This is not a course of toy notebooks; it culminates in an enterprise-grade capstone. You will choose from a slate of realistic briefs—among them a *clinical-trial match-maker*, a *compliance sentinel*, an *earnings analyst*, a *procurement analyst*, a *pharmacovigilance monitor*, a *newsroom verifier*, and a *legislative cartographer*—each drawn from a domain where retrieval quality has real consequences.

The capstone is where the fourteen instruments come together on a corpus you do not control, with messy documents, adversarial questions, and a stakeholder who wants to know whether the thing actually works. Start thinking now about which domain pulls at you; the briefs live in the capstone chapter.

We mention the capstone here, on day one, for a reason. Every concept we meet—every chunking strategy, every re-ranker, every guardrail, every metric—should be read with your eventual project in mind. The geometry we studied today is not trivia; it is the ground on which your system will stand or fall. Keep one foot in the theory and one eye on the build.

Let Us Do Some Exercises

WE CLOSE THE DAY WITH OUR HANDS. In the notebooks you will: embed a small corpus and inspect the distribution of pairwise cosine similarities (and watch concentration appear before your eyes); contrast a *static* and a *contextual* embedding on a polysemous word like “bank” and see the second resolve what the first cannot; compare lexical and semantic retrieval on queries designed to make each one fail; load two embedding models—one raw, one trained for retrieval—into the projector and measure how separated the three subjects become under each; and, for the curious, point LARQL at a small open model and ask it to DESCRIBE and TRACE a fact you care about.

Do not aim for polish today. Aim to *feel* the phenomena: the spike in the similarity histogram, the funnel in the projector, the two “bank” vectors flying apart, the moment a keyword query finds what a semantic query missed. Felt phenomena are the ones you remember when you are debugging at midnight in Week 9.

These exercises are deliberately diagnostic rather than constructive. We are not building a system yet; we are building *intuition you can trust*. Next week, with that intuition in hand, we make our first real engineering decision—where to cut.

Essential Readings

THESE ARE REQUIRED. Each is a load-bearing pillar of the field, chosen for historical significance and for the role it plays in the worldview we built today. Read them for the argument, not for every equation.

- **Bahdanau, Cho & Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate***
The *original* attention paper. Read it to see attention born as a cure for the seq2seq bottleneck—soft alignment between input and output—before it became an architecture in its own right.
- **Vaswani et al., *Attention Is All You Need***
The transformer. The discovery that let representations scale, and the engine beneath every model we will use. Read it as the thesis that attention, alone, suffices—and study the scaled dot-product equation until the query/key/value lookup feels natural.
- **Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers***
Establishes encoder-based, bidirectional, *contextual* representation learning—the modern embedding paradigm and the resolution to polysemy. This is the model whose raw geometry we saw funnel in the projector.
- **Reimers & Gurevych, *Sentence-BERT***
Why we do not use raw BERT for retrieval. Sentence-BERT shows how to produce sentence embeddings whose cosine similarity is actually meaningful—the move from a funnel toward separated clusters.
- **Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks***
The paper that named RAG. Read it as the architectural response to the limits of parametric memory—the thesis of this entire course in its original form.
- **Gao et al., *Retrieval-Augmented Generation for Large Language Models: A Survey***
The single best map of the territory. Skim it now for vocabulary and the naive/advanced/modular taxonomy; return to it after each module to see where the week’s instrument fits.

Oliver Twist’s List of Readings

OPTIONAL, FOR THE VORACIOUS MIND. Each sharpens judgment rather than adding surface familiarity. None is required for Week 2.

- Mikolov et al., *Efficient Estimation of Word Representations*, and Pennington et al., *GloVe*

The two classic *static* embeddings: the first vivid demonstrations that semantic relationships live as *directions* in vector space—and, by their single-vector-per-word design, the clearest illustration of why polysemy demanded contextual successors.

- Ethayarajh, *How Contextual Are Contextualized Word Representations?*

The careful study of anisotropy we leaned on today—why raw contextual embeddings occupy a narrow cone, measured rather than asserted.

- Gao et al., *SimCSE*

A first, gentle answer to the question we deliberately left open: how training can spread a funnel into a separated, more isotropic space. Read it when the “why?” starts to itch.

- Karpukhin et al., *Dense Passage Retrieval*

How embedding-based retrieval is actually trained for question answering—the bridge from word vectors to a working bi-encoder retriever.

- Elhage et al., *Toy Models of Superposition*, and Bricken et al., *Towards Monosemanticity*

The mechanistic-interpretability companion to our discussion of polysemy: why single neurons represent many features, and how dictionary learning tries to pull them apart.

- Hayuk, *LARQL*

Hands-on mechanistic interpretability: decompile a small model into a queryable vindex and browse, trace, and edit its knowledge—“the model is the database.” The most direct way to feel that a model’s knowledge is geometry.

- Vershynin, *High-Dimensional Probability*, and Indyk & Motwani, *Approximate Nearest Neighbours*

For those who want the mathematics of concentration of measure, and the computational reality of similarity search under the curse of dimensionality, done properly.

A Calibration Note

TODAY WAS DELIBERATELY EXPANSIVE, and if some of it felt premature, that is by design. We did not aim for mastery; we aimed for orientation. We watched semantic search do the impossible, traced it to the discovery of attention and the rise of the transformer, established that machines reason in vectors rather than words, met the problem of polysemy and its resolution through context, glimpsed that a model's own knowledge is queryable geometry, and then looked hard at the space those vectors live in—its curse, its concentration, its anisotropy. Finally we built the simplest RAG system and watched the geometry predict, line by line, how it breaks.

Do not mistake breadth for vagueness. Every idea introduced today returns, sharpened and instrumented, in a later week. The funnel you saw in the projector becomes a chapter on choosing and adapting embedders. The thin margin above the concentration noise floor becomes the re-ranking cascade. The lost-in-the-cut pathology becomes the chunking chapter. The polysemy collision becomes part of why we transform queries and preserve context. The map was not decoration; it was an argument, and you now hold its first premise.

Meaning has become geometry—in the embedding space, and in the weights themselves. Search is the art of moving well through that geometry, and every instrument we build this summer is a response to a quirk of the space we met today. Learn to see the space, and the rest of the course will feel less like a syllabus and more like common sense.

Next week: we make our first irreversible engineering decision. Before we can retrieve a thought, we must decide how to cut it from its document—and we will discover that the size of a thought is harder to measure, and more consequential, than it first appears.

References

- bahdanau2014attention Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio (2015). “Neural Machine Translation by Jointly Learning to Align and Translate”. In: **Proceedings of the 3rd International Conference on Learning Representations (ICLR)**. The original attention mechanism, introduced as soft alignment for sequence-to-sequence translation. arXiv: 1409.0473 [cs.CL].
- bricken2023monosemanticity Bricken, Trenton et al. (2023). “Towards Monosemanticity: Decomposing Language Models with Dictionary Learning”. In: *Transformer Circuits Thread*. Anthropic. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- devlin2018bert Devlin, Jacob et al. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of NAACL-HLT*.
- elhage2022superposition Elhage, Nelson et al. (2022). “Toy Models of Superposition”. In: *Transformer Circuits Thread*. Anthropic. https://transformer-circuits.pub/2022/toy_model/index.html.
- ethayarajh2019contextual Ethayarajh, Kawin (2019). “How Contextual Are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 55–65.
- gao2021simcse Gao, Tianyu, Xingcheng Yao, and Danqi Chen (2021). “SimCSE: Simple Contrastive Learning of Sentence Embeddings”. In: **Proceedings of EMNLP**.
- gao2023retrieval Gao, Yunfan et al. (2023). “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint arXiv:2312.10997*.
- larql2025 Hayuk, Chris (2025). **LARQL: Query Neural Network Weights Like a Graph Database**. <https://github.com/chrishayuk/larql>. Decompiles a transformer into a queryable *vindex*; the *Lazarus Query Language* (LQL) browses, traces, and edits a model’s knowledge. “The model IS the database.”

- indyk1998approx Indyk, Piotr and Rameev Motwani (1998). "Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality". In: **Proceedings of the 30th Annual ACM Symposium on Theory of Computing**, pages 604–613.
- karpukhin2020dpr Karpukhin, Vladimir et al. (2020). "Dense Passage Retrieval for Open-Domain Question Answering". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781.
- lewis2020rag Lewis, Patrick et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems* 33.
- mikolov2013efficient Mikolov, Tomas et al. (2013). "Efficient Estimation of Word Representations in Vector Space". In: *arXiv preprint arXiv:1301.3781*.
- pennington2014glove Pennington, Jeffrey, Richard Socher, and Christopher D. Manning (2014). "GloVe: Global Vectors for Word Representation". In: **Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)**, pages 1532–1543.
- radford2021clip Radford, Alec et al. (2021). "Learning Transferable Visual Models From Natural Language Supervision". In: *Proceedings of the 38th International Conference on Machine Learning*.
- reimers2019sbert Reimers, Nils and Iryna Gurevych (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992.
- sutskever2014seq2seq Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le (2014). "Sequence to Sequence Learning with Neural Networks". In: *Advances in Neural Information Processing Systems* 27.
- vaswani2017attention Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems* 30.
- vershynin2018highdim Vershynin, Roman (2018). **High-Dimensional Probability: An Introduction with Applications in Data Science**. Cambridge University Press.

There were 22 pages in this document.

FEEDBACK

Please send any feedback on this document, or report errors to the author at asif@supportvectors.com

